

VAE による画像生成

課題 7：変分オートエンコーダの実装と評価

B4 輪講 / M1 余吾純之介

発表の概要

授業で学んだ **VAE** を実装し、**MNIST 手書き数字**の再構成・生成を行った。

課題	内容
7-1	VAE の実装 (encoder・reparametrization_trick・decoder・kld・forward)
7-2	実行ごとに変わる結果を再現可能にする
7-3	z_dim / h_dim / drop_rate / lr を変えて結果を比較

到達目標

- ① 変分推論が必要な理由を説明できる
- ② VAEにおけるELBO の意味と損失の導出を説明できる
- ③ Reparametrization trick とその性質について説明できる

1. VAE の目的と変分推論

未知データを生成する確率モデル

VAE の目的と潜在変数モデル

目的：未知データの生成

訓練データの分布 $p(x)$ を近似し、新しいデータを生成する。

- x は**潜在変数 z から生成**されると仮定
- $z \sim p(z)=N(0, I) \rightarrow$ デコーダ $\rightarrow x$

計算上の課題

周辺尤度は積分で解析的に解くには限界がある。

$$p(x) = \int p(x|z) p(z) dz$$

事後分布 $p(z|x)$ も計算困難

→ 近似事後分布 $q(z|x)$ を導入
(**変分推論**)

2. ELBO と損失関数

対数周辺尤度の下界を最大化する

ELBO の導出

対数周辺尤度は **ELBO** $\mathcal{L}$ と **KL ダイバージェンス** に分解できる。〔式 (2)〕

$$\log p_{\theta}(x) = D_{KL}(q_{\phi}(z|x) \parallel p_{\theta}(z|x)) + \mathcal{L}(\theta, \phi; x)$$

- $D_{KL} \geq 0$ より $\log p_{\theta}(x) \geq \mathcal{L}(\theta, \phi; x)$... ELBO は対数周辺尤度の**下界**〔式 (3)〕
- $\log p_{\theta}(x)$ は直接最大化できない → **ELBO を最大化** (q_{ϕ} が真の事後分布に近づく)

ELBO を変形 〔式 (7)〕：

$$\mathcal{L} = \underbrace{\mathbb{E}_{q_{\phi}}[\log p_{\theta}(x|z)]}_{\text{再構成項}} - \underbrace{D_{KL}(q_{\phi}(z|x) \parallel p_{\theta}(z))}_{\text{正則化項}}$$

損失 = $-\mathcal{L}$ (VAE = AE + 正則化)

Kingma & Welling, *Auto-Encoding Variational Bayes*, arXiv:1312.6114 (2014). 式番号は同論文に対応。

損失関数の具体形

再構成項 (Appendix C.1)

$$\log p_{\theta}(x|z) = \sum_{d=1}^D x_d \log y_d + (1-x_d) \log(1-y_d)$$

各画素を Bernoulli に分布すると仮定 (0=黒, 1=白)。 y はデコーダの Sigmoid 出力。

正則化項 (Appendix B)

$$-D_{KL}(q_{\phi}(z|x) \parallel p_{\theta}(z)) = \frac{1}{2} \sum_{j=1}^J (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

$q_{\phi}=N(\mu, \sigma^2 I)$, $p_{\theta}=N(0, I)$ より
積分が閉形式で解ける。

Reparametrization Trick

問題：サンプリングは微分不可

$z \sim N(\mu, \sigma^2)$ を直接サンプリングする操作は微分できない。

-> **エンコーダに勾配が伝わらない。**

解決：Reparametrization trick (Section 2.4・式 (4))

ノイズ ε を外からサンプリングし、 z を決定論的な関数で表す。

$$z = g_\phi(\varepsilon, x) = \mu + \varepsilon \odot \exp\left(\frac{1}{2} \log \sigma^2\right), \quad \varepsilon \sim N(0, I)$$

z が μ, σ で微分可能になり、**デコーダからエンコーダまで誤差逆伝播が可能。**

3. 実装

VAE_skeleton の #TODO 5 箇所を実装

#TODO ① **encoder** : 画像 $\rightarrow (\mu, \log \sigma^2)$

入力画像 x を全結合層に通し、近似事後分布 $q_\phi(z|x) = N(\mu, \sigma^2 I)$ のパラメータを出力。

```
def encoder(self, x: torch.Tensor):
    x = x.view(-1, self.x_dim)
    # TODO: enc_fc1  $\rightarrow$  ReLU  $\rightarrow$  enc_fc2  $\rightarrow$  ReLU の順に通す。
    #       その後 enc_fc3_mean と enc_fc3_logvar に通して mean と log_var を返す。
    h = torch.relu(self.enc_fc1(x))    # 784  $\rightarrow$  h_dim
    h = torch.relu(self.enc_fc2(h))    # h_dim  $\rightarrow$  h_dim//2
    mean    = self.enc_fc3_mean(h)     # h_dim//2  $\rightarrow$  z_dim
    log_var = self.enc_fc3_logvar(h)   # h_dim//2  $\rightarrow$  z_dim (同じ h から分岐)
    return mean, log_var
```

同じ中間表現 h から μ と $\log \sigma^2$ に分岐。分散は負を避けるため $\log \sigma^2$ で出力する。
[Appendix C.2]

#TODO ② reparametrization_trick : z のサンプリング

サンプリングを「外部ノイズ ε + 決定論的演算」に分解し、微分可能にする。

```
def reparametrization_trick(
    self, mean: torch.Tensor, log_var: torch.Tensor
) -> torch.Tensor:
    # TODO: mean と同じ形状の  $\varepsilon$  を標準正規分布からサンプリングし、 $z$  を計算して返す。

    eps = torch.randn_like(mean)          #  $\varepsilon \sim N(0, I)$ 
    z = mean + eps * torch.exp(0.5 * log_var) #  $z = \mu + \varepsilon \odot \sigma$ 
    return z
```

$z = \mu + \varepsilon \odot \exp(\frac{1}{2} \log \sigma^2)$ 。勾配が μ, σ を通りエンコーダへ伝播する。
[Section 2.4, Eq. (4)]

#TODO ③ **decoder** : $z \rightarrow$ 再構成画像

潜在変数 z から画像を復元。最終層 Sigmoid で各画素を $[0, 1]$ に収める。

```
def decoder(self, z: torch.Tensor) -> torch.Tensor:
    # TODO: dec_fc1 → ReLU → dec_fc2 → ReLU → dec_drop → dec_fc3 → Sigmoid の順に通す。
    h = torch.relu(self.dec_fc1(z))    # z_dim → h_dim//2
    h = torch.relu(self.dec_fc2(h))    # h_dim//2 → h_dim
    h = self.dec_drop(h)                # Dropout は dec_fc3 の直前
    y = torch.sigmoid(self.dec_fc3(h))  # h_dim → 784
    return y
```

出力 y は Bernoulli 尤度 $p_{\theta}(x|z)$ のパラメータ（画素ごとの「白の確率」）。
[Appendix C.1]

#TODO ④ `kld` : KL ダイバージェンス (解析解)

$q_\phi = N(\mu, \sigma^2 I)$, $p_\theta = N(0, I)$ の KL を閉形式で計算。

```
def kld(self, mean: torch.Tensor, log_var: torch.Tensor) -> torch.Tensor:
    # TODO: KL[q(z|x) || p(z)] の解析解を実装する。
    # torch.distributions.kl_divergence() の使用は禁止。
    kl = -0.5 * torch.sum(
        1 + log_var - mean.pow(2) - torch.exp(log_var), dim=1
    )
    return kl.sum() # バッチ方向に合算してスカラーに
```

$D_{KL} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$ をそのまま実装。

`dim=1` で潜在次元を和、`.sum()` でバッチを合算。

[Appendix B]

#TODO ⑤ forward : ELBO の構成

encoder → reparam → decoder を呼び、ELBO の 2 項 (KL・再構成) を返す。

```
def forward(self, x: torch.Tensor):
    # TODO: encoder → reparametrization_trick → decoder の順に呼び出す。
    #       elbo_kl = -self.kld(mean, log_var) で KL 項と (符号に注意)、
    #       elbo_rec を計算して [elbo_kl, elbo_rec], z, y を返す。
    x = x.view(-1, self.x_dim)
    mean, log_var = self.encoder(x)
    z = self.reparametrization_trick(mean, log_var)
    y = self.decoder(z)
    elbo_kl = -self.kld(mean, log_var) # 符号反転で  $\leq 0$  にする
    # Bernoulli 対数尤度 (Appendix C.1 Eq.11)
    elbo_rec = torch.sum(
        x * torch.log(y + self.eps) + (1 - x) * torch.log(1 - y + self.eps),
        dim=1,
    ).sum()
    return [elbo_kl, elbo_rec], z, y
```

7-2 再現性の確保

問題： `main.py` は実行ごとに結果が変わる。

乱数が固定されていない箇所：

重み初期化 / train/val 分割 (`random_split`) / バッチ shuffle / Reparam の ϵ (`randn`)

解決： 学習・データ生成より前に乱数シードを固定する。

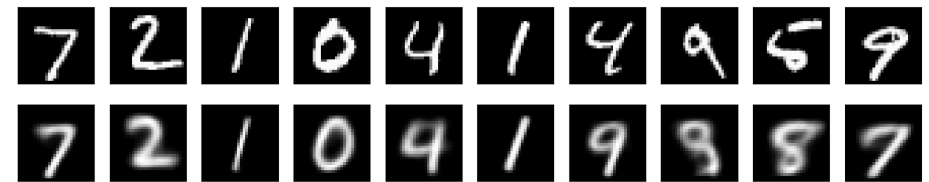
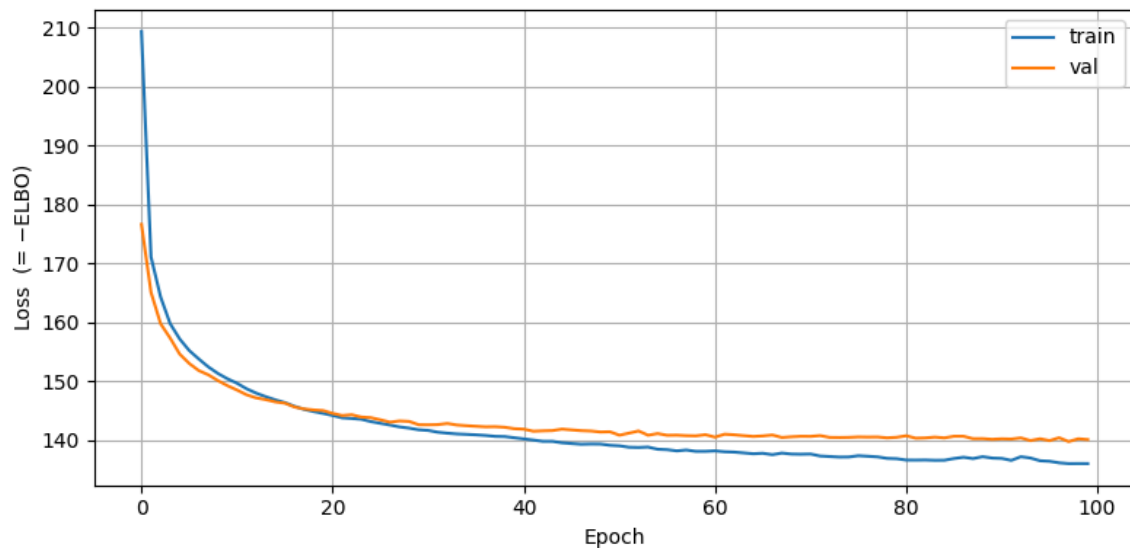
```
torch.manual_seed(42)
torch.cuda.manual_seed(42)
```

検証： 同条件で 2 回実行し、Loss が完全一致することを確認した。

4. 実験結果

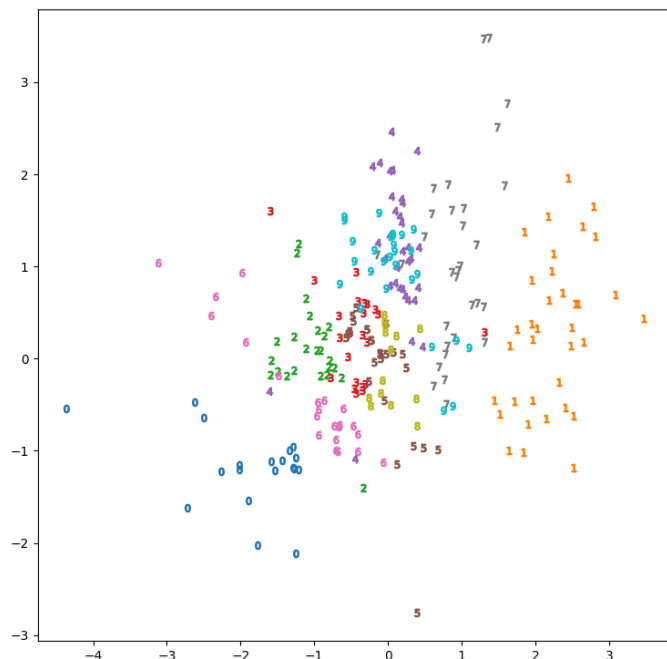
Loss の収束・再構成・潜在空間

Loss 推移と再構成 (z_dim = 2)



上段=入力 / 下段=再構成

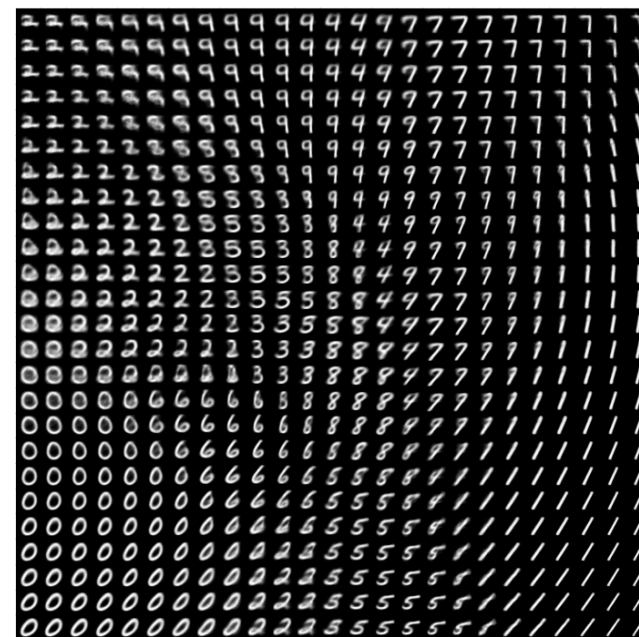
train / val とも単調に減少して収束 (val $\approx$ 140)。
過学習は見られず、入力の数字に**追従して再構成**できている。



潜在空間 (散布図)

数字クラスごとに塊が分離。

正則化により $N(0, I)$ 付近へ整列。

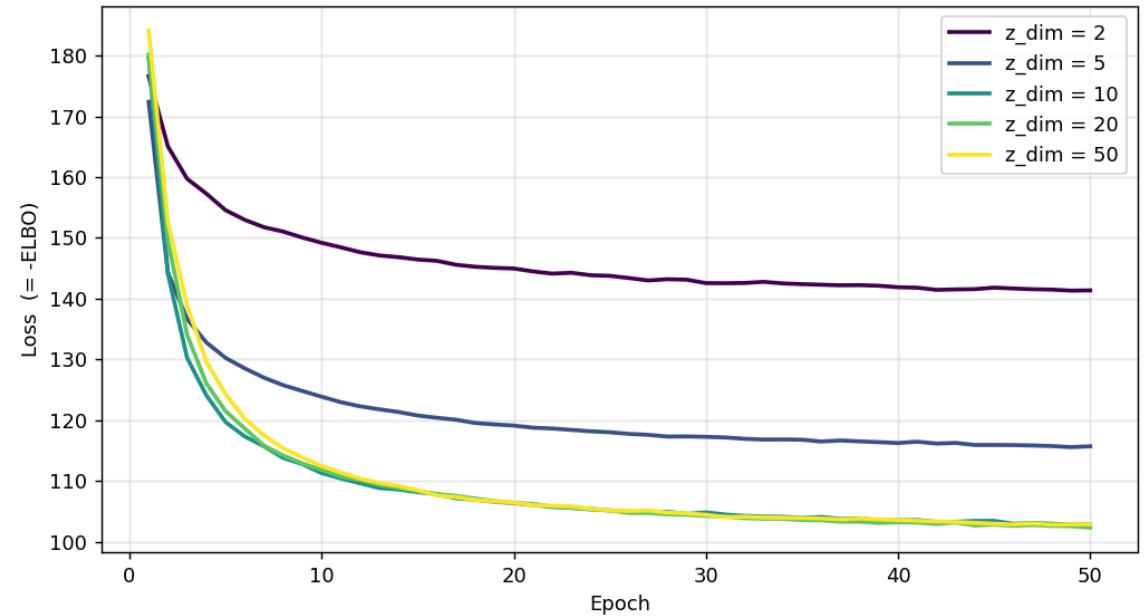
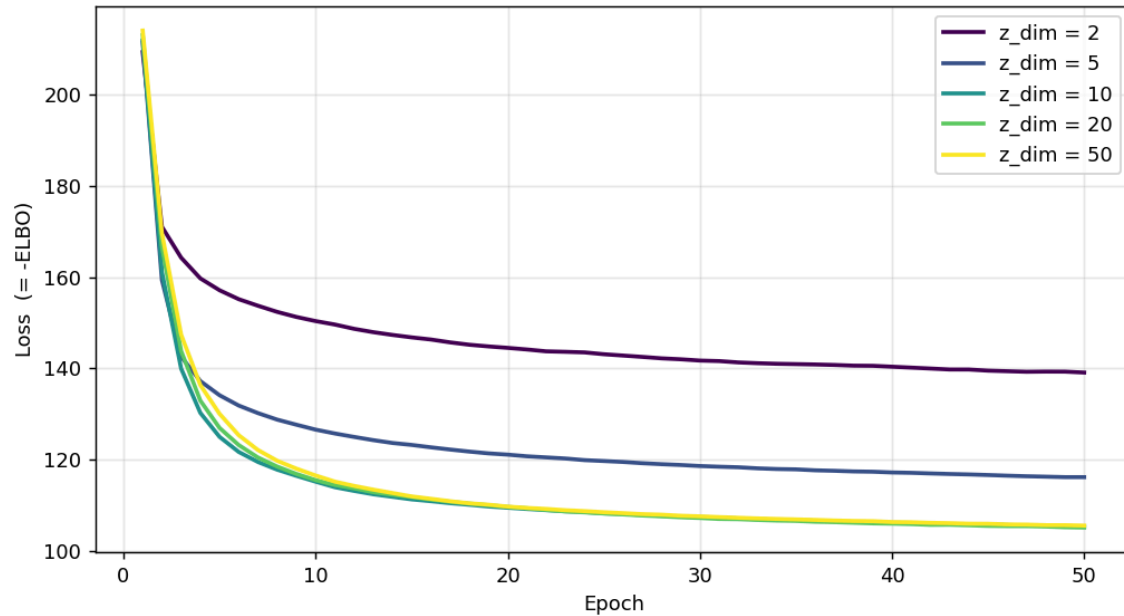


格子点からの生成

数字が連続的に変化する滑らかな多様体。

隣接する z が似た字形を生成。

7-3 実験①：潜在次元 z_dim



左：train sweep / 右：val sweep ($z = 2/5/10/20/50$ 、他のパラメータはデフォルトで固定, epochs=50)

z_dim を増やすほど Loss は低下（表現力↑）。val は **141** → **103** で改善後ほぼ頭打ち。

再構成の鮮明さと潜在次元 z_dim の関係性

20

$z = 2$ （上=入力 / 下=再構成）



$z = 10$

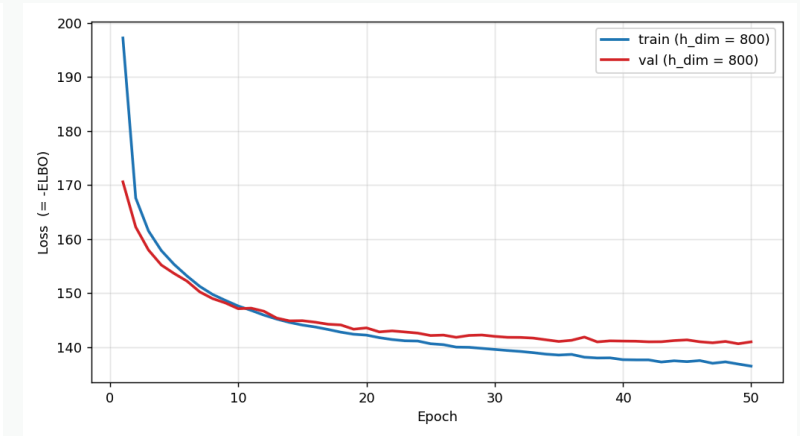
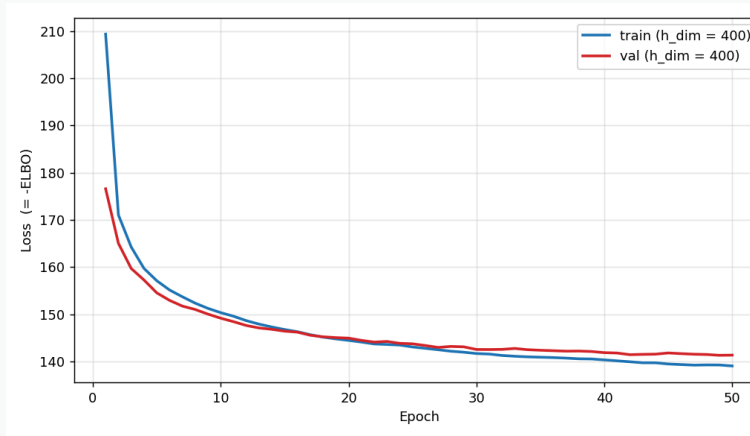
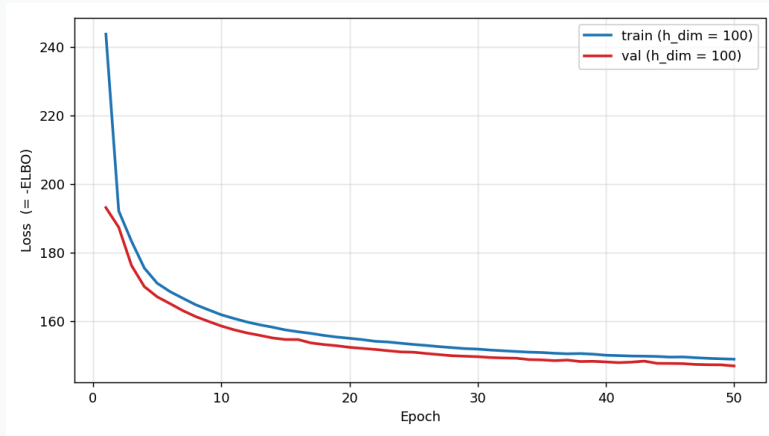


$z = 50$



z_dim を上げるほど再構成が鮮明になり、再構成誤差も低下する。
デコーダは期待値（平均像）を出すため生成はややぼやける。

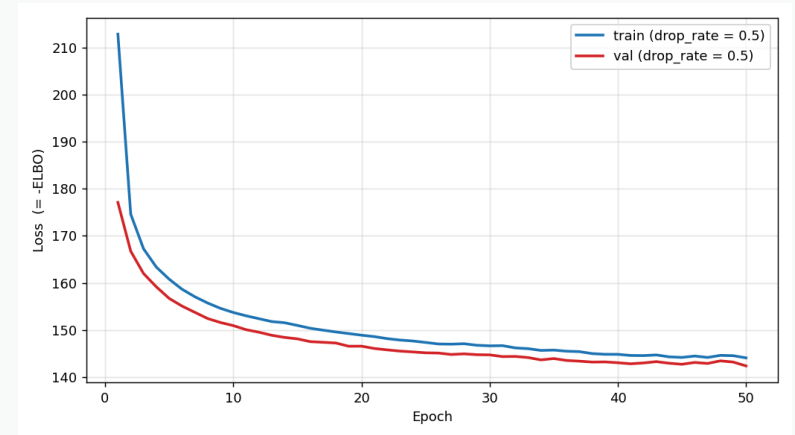
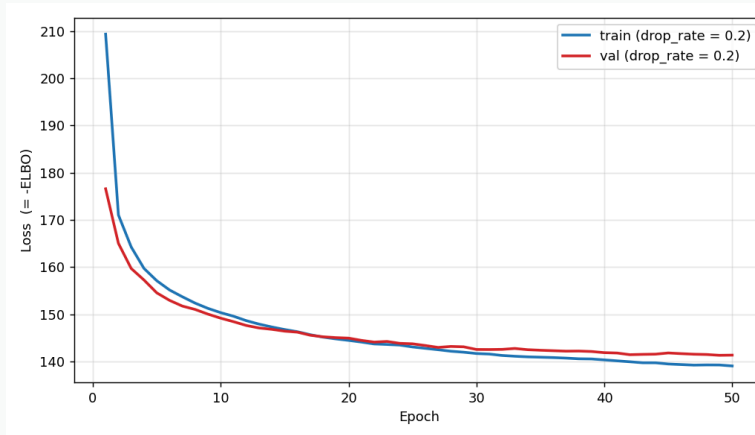
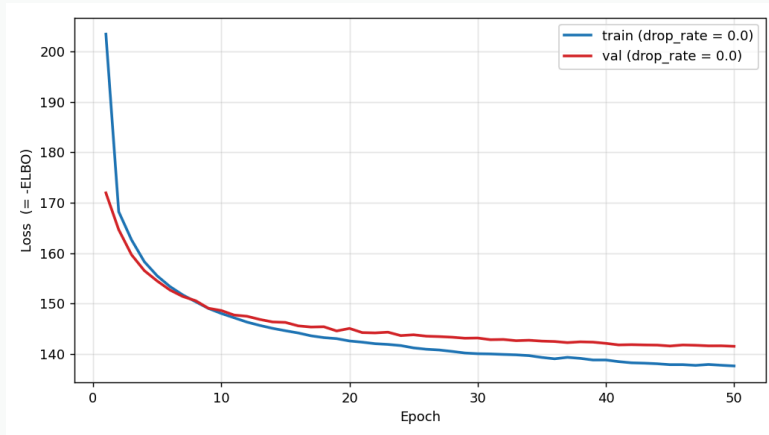
7-3 実験②：中間層 $h_dim \in \{100, 400, 800\}$



$h_dim = 100 / 400 / 800$ (他のパラメータはデフォルトで固定, , epocs=50)

中間層を広げるほど **Encoder/Decoder の表現力が向上**し再構成誤差（train）は低下。
一方 **trainとvalの間のギャップが拡大**し、800 では val が下げ止まる＝**過学習の兆候**。

7-3 実験③：Dropout 率 $\text{drop_rate} \in \{0.0, 0.2, 0.5\}$

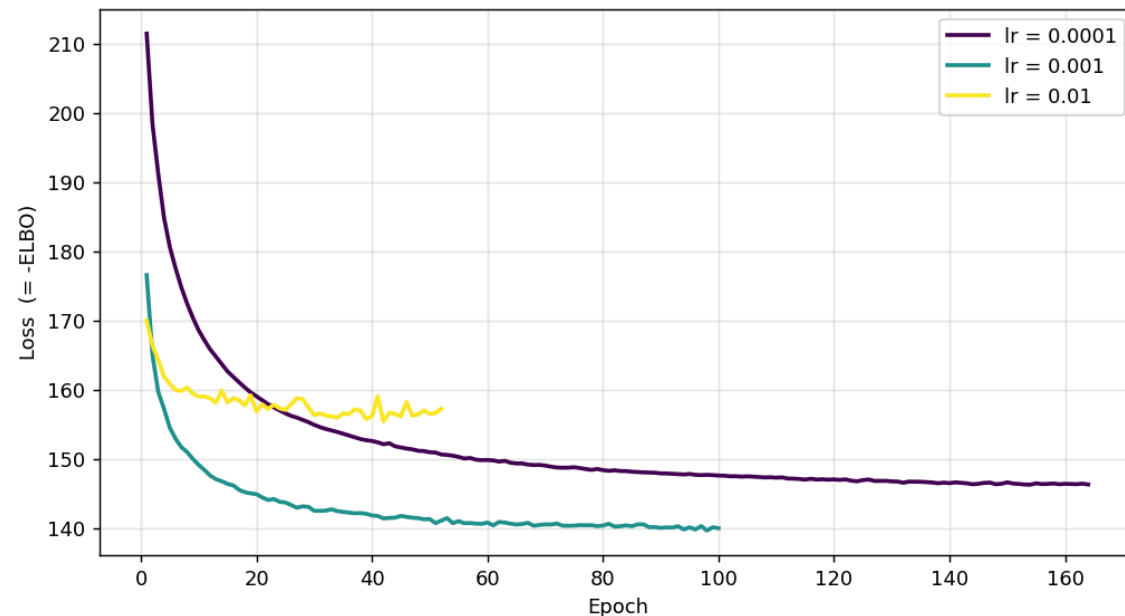
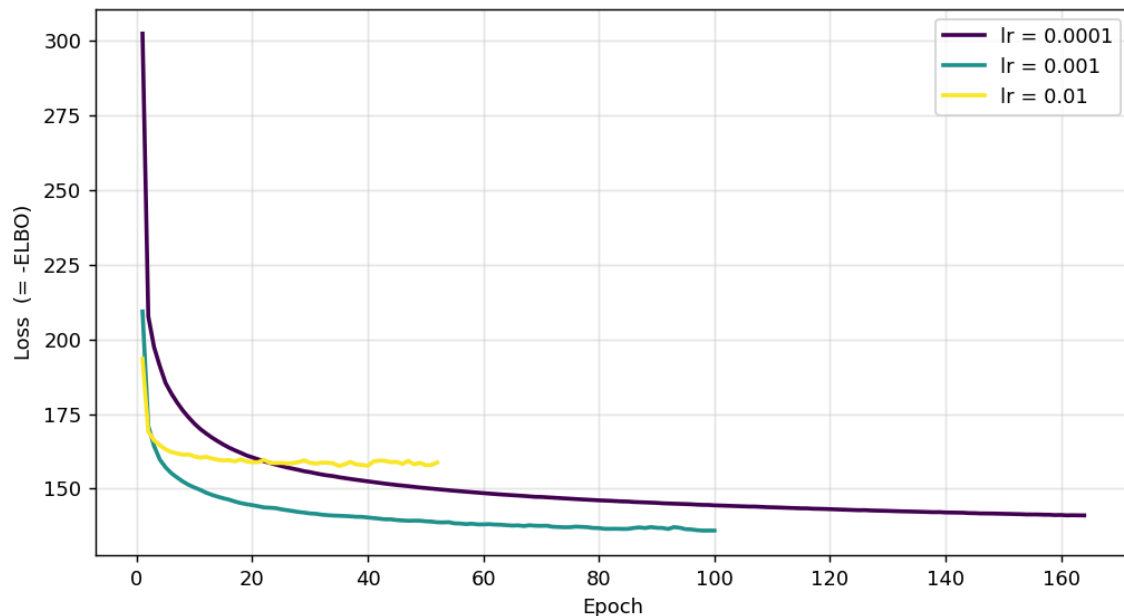


$\text{drop_rate} = 0.0 / 0.2 / 0.5$ (他のパラメータはデフォルトで固定, , epochs=50)

- 0.0**：train が val を一貫して下回っており、**軽微な過学習**が起きている
- 0.2**：train と val がほぼ密着したまま収束しており、過学習が最も抑制されている
- 0.5**：正則化が過剰で train と val が逆転（**アンダーフィット**）

7-3 実験④：学習率 $lr \in \{0.01, 0.001, 0.0001\}$

23



左：train sweep / 右：val sweep / $lr = 0.01/0.001/0.0001$

0.01：収束は速いが学習曲線が**振動**し、early stopping が ep52 で発動

0.001：最良 (val $\approx$ 140、ep98 で最小)

0.0001：安定だが**収束が遅く**、val $\approx$ 146 に達するのに約 150ep を要する。

→ **収束速度と安定性のトレードオフ。**

まとめ：3つの到達目標

① なぜ変分推論が必要か

周辺尤度 $p(x) = \int p(x|z)p(z) dz$ と事後分布 $p(z|x)$ は解析的に解けない。

→ 計算できる近似事後分布 $q(z|x)$ を導入し、積分を最適化問題に置き換える。

② ELBO の意味と損失

$\log p(x) = \mathcal{L} + D_{KL}(q||p(z|x)) \geq \mathcal{L}$ 。解けない $\log p(x)$ の代わりに下界 $\mathcal{L}$ を最大化する。

損失 $= -\mathcal{L} =$ 再構成誤差 (Bernoulli 対数尤度) $+$ 正則化項 (KL)。

③ Reparametrization trick

$z \sim N(\mu, \sigma^2)$ の直接サンプリングは微分不可 $\rightarrow z = \mu + \varepsilon \odot \sigma$, $\varepsilon \sim N(0, I)$ に分離。

ノイズを外に出すことで z が μ, σ で微分可能になり、Encoder まで誤差逆伝播できる。

まとめ：実装と実験

課題	取り組み
7-1	encoder・reparametrization_trick・decoder・kld・forward を実装
7-2	乱数シード固定
7-3	z_dim / h_dim / drop_rate / lr を比較

今後の課題

- ハイパーパラメータの**同時最適化**
- t-SNE・PCA による高次元潜在空間の可視化
- FashionMNIST や音声合成（VAE-SiFiGAN）への応用